ConceptFlow: A Source-Grounded, Locally Hosted Adaptive Study Companion for Multimodal Course Material

Palak Mittal
Data Science and AI
IIT Mandi, India
b25220@students.iitmandi.ac.in

***Abstract*—Students study from lecture recordings, textbooks, slides and personal notes distributed across different sources. ConceptFlow integrates these materials into a local web application for cited explanations, revision resources and assessments. Format-specific readers, a vision model and speech recognition produce passages with page, slide or timestamp metadata. BM25 retrieves evidence for a local language model, while the application resolves citations from stored source records. Bayesian Knowledge Tracing maintains per-topic learner estimates from assessment responses and grounded study activity. Account-specific storage preserves workspaces, chats, generated resources and reports. This report presents the architecture, grounding method, learner model, setup and measured development results. A 12-question retrieval baseline achieved a 75.00% top-one hit rate and a 100.00% top-three hit rate on the bundled graph notes. Controlled simulations demonstrate learner-state updates and weak-topic selection. These bounded results describe prototype behaviour and do not establish educational gains for real students.**

***Index Terms*—adaptive learning, retrieval-augmented generation, Bayesian Knowledge Tracing, source citation, multimodal ingestion, local inference**

I. Introduction

Exam preparation often involves lecture recordings, a textbook, slide decks and handwritten notes. Searching across these resources takes time, and a general chatbot may provide an explanation without identifying relevant course evidence or considering the student's prior activity.

ConceptFlow combines material ingestion, cited tutoring, revision tools, assessments and progress views. Extracted passages retain their source locations; pretrained text, vision and speech models run locally; and an explicit learner model maintains topic estimates separately from generated explanations. The project integrates existing models and application components rather than training a new foundation model.

II. System Architecture

ConceptFlow uses a React frontend and a FastAPI backend. Materials supports uploads and workspaces; Learn provides cited tutoring; Practice offers preset and customised assessments; Progress displays outcomes and learner estimates; and History preserves chats and study resources. Figure 2 shows the system architecture.

Each account has its own SQLite database, upload directory and cache. Request-local account context routes operations to the signed-in user's resources, and background workers retain the workspace context in which their jobs began. Upload reception is separated from reading and indexing. Long operations expose progress through polling and support explicit cancellation at processing checkpoints. Study screens retain their state when the user switches between application pages.

Ingestion, tutoring and assessment share source metadata and local inference services. Original materials and generated artifacts are stored separately. Archived sources and workspaces retain their saved chats, assessments and study resources.

III. Knowledge Base Construction

A. Source-Linked Segments

Each reader emits text segments with an origin: a PDF page, slide number, recording timestamp, spreadsheet row or code-line range. A chunker splits long segments into retrieval-sized units while preserving citation metadata. SQLite stores these units and source records; original files remain in private local storage.

B. Multimodal Reading

Figure 1 shows the project workflow. Text-layer PDFs use PyMuPDF and pypdf, slides use python-pptx, and spreadsheets use Calamine. Qwen3-VL 2B Instruct [1] reads scanned pages, photographs and handwritten notes. Image quality and notation clarity affect extraction accuracy.

Audio and video speech is decoded with PyAV and transcribed using faster-whisper [2] with Whisper Base [3]; the default uses CPU execution, int8 computation and beam size one. YouTube imports first try timestamped captions, then use audio transcription when usable captions are absent. Public document and website imports use the background reading workflow. Materials retains the transcript feature for audio, video and YouTube sources. Video ingestion currently reads speech; on-screen-only information is outside this path.

C. Processing and Caching

Successful vision readings, transcripts and reusable study drafts are cached using source content or prompts and model identity. Cached transcripts receive the correct metadata for each upload. The loaded speech model is reused, and inference

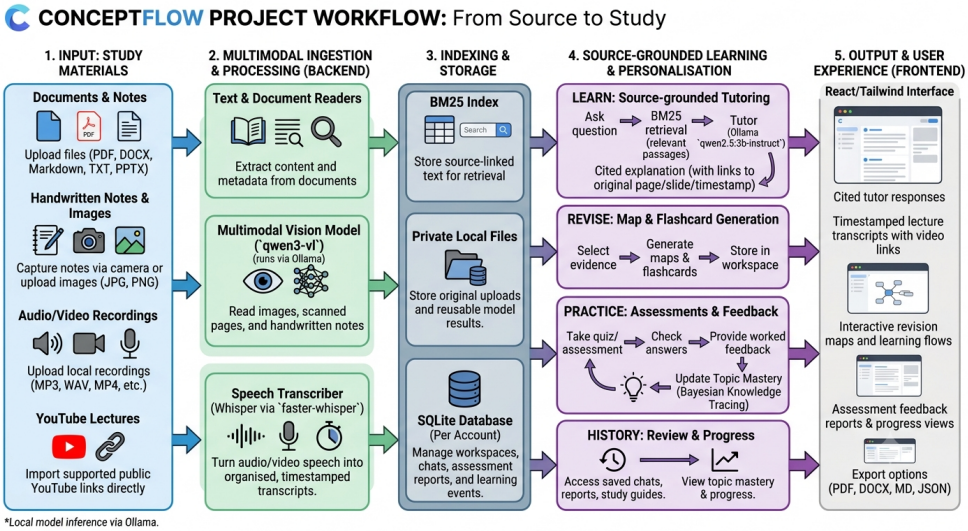


Fig. 1. Project workflow of ConceptFlow.

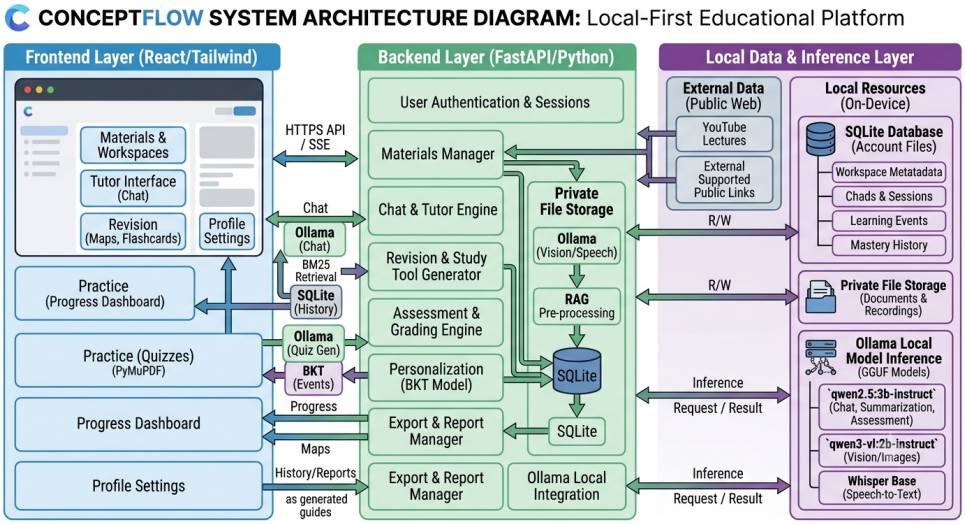


Fig. 2. System architecture of ConceptFlow.

is serialised to avoid CPU contention. Guide sections and prerequisite-link results are reused when their inputs match. Caching reduces repeat work; first-run processing depends on material length, complexity and hardware.

IV. Source-Grounded Tutoring

A. Retrieval and Citations

BM25 [4] ranks indexed passages against a question. Evidence and learner context are supplied to Qwen2.5 3B Instruct [5], served by Ollama [6]. This follows retrieval-augmented generation [7] using lexical retrieval.

Citation links come from stored page, slide and timestamp metadata. This preserves navigation, although a correct link alone does not prove that every associated claim is supported. Full-material lessons process evidence in source order and batches. Insufficient material is surfaced through an evidence-gap notice or refusal.

B. Lesson Checks and Revision

The lesson pipeline checks for truncation and repetition and compares generated sections with evidence in a separate model call. This uses the same local model as generation, so the check is heuristic. Failed sections can show labelled source excerpts. Learning flows suggest a study sequence; revision maps offer a topic overview; and flashcards support recall. Accepted guide batches can serve multiple revision views.

C. Language and Presentation

English is the default. Hindi and Romanised Hinglish are experimental options with retry and language-check notices. Excerpts and transcripts retain their source language. KaTeX

renders mathematical expressions. PDF, DOCX, Markdown, TXT and JSON exports are available for supported study artifacts.

V. Assessment and Learner Model

A. Question Generation and Format Selection

The student chooses source materials, topic scope, difficulty, count and formats. Preset styles and the custom builder share the generation pipeline. Supported formats are multiple choice, short answer, numerical, true/false and fill-in-the-blank.

Small batches reduce long structured-output failures. Candidates are checked against evidence, and valid questions are retained when another candidate fails. Application-generated IDs prevent collisions. Exact fingerprints reduce duplicate wording, while distinct questions may revisit the same concept. A selected mixture of formats is assigned a target distribution.

When AI drafting cannot supply enough usable candidates, the system can add explicitly labelled source-recall exercises with answer keys copied from selected passages. Numerical recall asks for stated values; it is not a substitute for verified multi-step calculations. If the count cannot be supported, the saved paper identifies the shortfall. Previously used recall questions may be reused with a notice when the remaining pool is exhausted.

B. Scoring and Misconception Feedback

Objective responses use deterministic comparison, with a tolerance for numerical answers. Written responses use a source-grounded rubric; nontrivial short answers are graded together in one model call. Blank and exact-answer responses can be handled directly. Reports retain responses, expected answers, explanations, citations, topic accuracy and weak-topic summaries.

Incorrect objective answers can receive misconception analysis. A proposed hypothesis must quote an exact excerpt from an allowed source and provide revision guidance. This checks provenance, not diagnostic truth. Blank responses and inconclusive analyses receive no label, and this analysis cannot change objective scores.

C. Bayesian Knowledge Tracing

BKT [8] maintains a topic's knowledge probability p. With slip s and guess g, the posterior after a correct answer is

$$p^* = \frac{p(1-s)}{p(1-s) + (1-p)g}. \tag{1}$$

After an incorrect answer,

$$p^* = \frac{ps}{ps + (1-p)(1-g)}. \tag{2}$$

A learning transition t produces

$$p' = p^* + (1-p^*)t. \tag{3}$$

The implemented parameters are $p_0 = 0.20$, $t = 0.15$, $s = 0.10$ and $g = 0.20$. These are heuristic defaults. A grounded tutoring exposure applies

$$p' = p + (1-p)\alpha, \qquad \alpha = 0.04. \tag{4}$$

TABLE I
Technology stack

Layer	Technologies
Frontend	React, JavaScript, Vite, Tailwind CSS, Lucide, KaTeX
Backend	Python, FastAPI, Pydantic
Storage	SQLite, SQLAlchemy, local files
Extraction	PyMuPDF, pypdf, Pillow, python-pptx, Calamine
Speech and links	faster-whisper, PyAV, yt-dlp
AI and learner	Ollama, BM25, BKT

TABLE II
Default inference models

Model	Purpose
Qwen2.5 3B Instruct	Explanations, guides, drafting, answer checks and rubric feedback
Qwen3-VL 2B Instruct	Images, scans and handwritten notes
Whisper Base	Timestamped speech transcription, CPU and int8

An exposure is not scored as a correct answer. New topics start from the prior, and a diagnostic practice style is available. A calibrated intake assessment is not established by prior initialisation alone. Fixed scores and changing mastery estimates are displayed separately.

D. Revision and Planning

Weak-topic signals drive further practice and revision recommendations. The dashboard combines assessment scores, topic performance, daily activity and learner estimates. Calendar events and reminders are separate from observed activity, so a scheduled session does not count as completed study. In-app reminders become visible when the application is active.

VI. Implementation

Table I lists the technologies and Table II lists the default models. Models are pretrained and used for local inference. Accounts use salted password hashes and revocable HTTP-only sessions. Code uploads are read without execution, SQLite imports are read-only, and archives have size and nesting limits. Profile settings provide study defaults, themes, font size and avatars. The documented deployment model is local use.

VII. Setup and Execution

A. Prerequisites

Install Python 3.11, Node.js 24 LTS, Git and Ollama. On Windows, use 64-bit Python and the x64 Visual C++ Redistributable for speech processing. FFmpeg on PATH provides broader media compatibility. Initial setup requires internet access and disk space for dependencies and model weights.

B. Installation and Launch

Clone the repository [9], open its root directory and run:

```
python scripts/setup.py
python scripts/start.py
```

Setup installs dependencies and downloads the text, vision and speech models. Keep Ollama running. The start script launches the API on port 8000 and the interface on port 5173. Open http://127.0.0.1:5173; Ctrl+C stops both servers. Alternative ports can be selected with:

```
python scripts/start.py --api-port 8011 --web-port
    5174
```

Default settings require no configuration file. Copy `.env.example` to `.env` for overrides. Original sample material is provided in `samples/`.

C. Student Workflow

Sign in, create a workspace and upload files or import a link in Materials. Use Learn for cited explanations, then generate revision resources or select a practice style. Complete an assessment, review its report and Progress, and reopen saved material in History.

VIII. Testing and Evaluation

A. Software Verification

The recorded verification on 15 October 2026 completed 183 backend tests and 16 frontend tests successfully. The production build also succeeded, with a non-fatal bundle-size warning. These are software checks, not overall answer-accuracy percentages. The run used Windows, Python 3.14.2, pytest 9.1.1 and Node.js 24.18.0. Python 3.11 remains the recommended setup version; the recorded run used the existing development environment, not a fresh clone or clean-device installation.

Coverage included upload reception while a slow reader remained active, citation preservation, explicit cancellation, caching, assessment formats, recall fallback and saved progress. An integration test completed a 12-question mixed-format recall paper and verified that all feedback was saved and Progress updated. Checks can be rerun with:

```
python -m pytest -q
npm --prefix web test
npm --prefix web run build
```

Raw test and build outputs, commands, runtime versions and the audited source commit are recorded under `docs/evaluation/`. The evidence runner `scripts/verify_report_evidence.py` regenerates these logs. Counts describe this recorded version and can change as tests are added or removed.

B. Requirement Coverage

Table III uses Done for an implemented and exercised mechanism and Partial for limited scope or incomplete validation. It does not equate implementation with proven educational effectiveness.

TABLE III
Requirement coverage

Requirement	Status	Implementation note
Video, textbook and slide ingestion	Done	Readers and workers; video speech and YouTube captions
Source-linked content	Done	Page, slide, timestamp, row or line metadata
Figures and diagrams	Partial	Image reading; limited specialist visual and video-frame coverage
Cited answers and navigation	Done	Links resolved from stored metadata
Tagged questions and novelty	Partial	Format targets, fingerprints and disclosed recall reuse
Answer verification	Partial	Source checks and same-model solving
Feedback and weak-topic reports	Done	Saved answers, citations and misconception hypotheses
Per-topic learner model	Done	BKT and tutoring-exposure updates
Diagnostic intake	Partial	Prior and diagnostic practice; no calibrated intake study
Revision and planning	Done	Maps, flows, flashcards, calendar and reminders
Language and audio	Partial	Experimental Hindi/Hinglish and browser narration
Framework RAG evaluation	Not measured	Custom retrieval diagnostics only; generation metrics remain unmeasured
Student simulations	Partial	Controlled BKT runs; no real-student learning measurement

TABLE IV
Evaluation evidence and remaining validation

Criterion area	Recorded evidence	Scope or remaining check
Operational reliability	183 backend tests; 16 frontend tests; build passed	Existing-environment regression checks, not all-format accuracy
Source retrieval	12 queries; Hit@3 100%; MRR 0.875	One graph sample, 15 passages; four negatives test retrieval only
Grounded answers	Citation preservation exercised; 4/4 zero-overlap queries retrieved nothing	RAGAS faithfulness, claim support and tutor refusal rates not measured
Assessment workflow	Five supported formats; one 12-question mixed-format recall integration test	Saved feedback and progress checked; independent question-quality review needed
Adaptive behaviour	Three profiles, 30 seeds each; 9,000 answer events across two policies	Fixed-ability simulation, not real-student learning gains
Novelty and repetition	Repetition 0.33% for novice-adaptive runs; 0% otherwise	Two DFS pool wraps explain all repeats; semantic novelty not measured
Processing efficiency	No reproducible latency benchmark reported	Cold/warm-cache, hardware and full-lecture timing remain to be measured

C. Summary of Results

Table IV connects the requirement areas to available quantitative or integration evidence. It complements feature coverage by distinguishing demonstrated behaviour from validation still needed.

D. Retrieval and Generation Evaluation

A reproducible retrieval baseline was run on 15 indexed passages from the unmodified bundled file `samples/graph-basics.md`. Twelve questions were paired with reference paragraph locations. BM25 used its implemented defaults, $k_1 = 1.5$ and $b = 0.75$, with three passages per query.

TABLE V
PRIMARY AND SUPPLEMENTARY RETRIEVAL DIAGNOSTICS

Set	n	Hit@1	Hit@3	MRR	P@3	R@3
Primary labelled set	12	75.00%	100.00%	0.875	41.67%	97.22%
Without paragraph 15	12	75.00%	100.00%	0.875	36.11%	100.00%
Paraphrased diagnostic	4	50.00%	75.00%	0.625	41.67%	66.67%

P@3 and R@3 denote macro Precision@3 and Recall@3; the paraphrased set is a small diagnostic, not held-out evaluation.

Hit@k indicates that at least one labelled reference passage occurs in the first k results. Precision@3 counts labelled references among the three results, and recall@3 divides that count by the labelled references. The primary 12 questions have 16 labelled references, so the maximum possible macro Precision@3 is $16/36 = 44.44\%$. Four additional zero-overlap queries returned no passages; this checks retrieval only, not end-to-end tutor refusal.

The observed Precision@3 of 41.67% is near its 44.44% ceiling; remaining result slots are unlabelled, so they cannot all be called irrelevant. MRR is 0.875; each of the three Hit@1 misses placed a labelled reference at rank two. Paragraph 15 is a revision-question section containing BFS review questions and answers. It appeared in the top three for 10 of 12 queries and was labelled as a reference for three; the other seven placements are unjudged. It does not address DFS, so it is not a reference for the DFS query. Removing the paragraph and its associated labels is a sensitivity check. Since this also changes the corpus and BM25 statistics, it does not prove the paragraph caused or inflated the hits.

Four vocabulary-overlap unsupported queries all retrieved at least one passage. This exposes a limitation of lexical retrieval; it does not test whether the tutor correctly refuses unsupported questions. Uniformly selecting three of 15 passages without replacement would have expected Hit@3 of about 25.49% for this reference-size mix; this is an analytic random baseline. Supplementary inputs, outputs and rerun instructions are in the repository at `docs/evaluation/supplement/`. The script records Python/platform information and SHA-256 hashes for the source notes, script and query cases, in the same provenance style as the primary benchmark. These custom retrieval diagnostics are not framework-based generation scores.

No generation-latency figure is reported because a retained reproducible timing record was unavailable. Framework-based generation metrics such as RAGAS [10] faithfulness and response relevance were not measured for the restored application version evaluated here. This remains an unmet evaluation requirement; the retrieval baseline is not a substitute.

E. Personalisation Evaluation

Three fixed-ability profiles were simulated across graph representations, BFS and DFS. Topic abilities were $(0.25, 0.30, 0.35)$ for the novice, $(0.55, 0.65, 0.75)$ for the intermediate and $(0.10, 0.85, 0.80)$ for the uneven profile.

TABLE VI
CONTROLLED SIMULATION MEANS OVER 30 SEEDS

Profile	Policy	Correct (mean ± SD)	Estimate change (mean ± SD)	Repeated
Novice	A	$41.73 \pm 6.01\%$	$+61.07 \pm 9.69$ pp	0.33%
Novice	R	$41.33 \pm 5.81\%$	$+39.77 \pm 13.32$ pp	0.00%
Intermediate	A	$63.67 \pm 7.68\%$	$+78.61 \pm 3.01$ pp	0.00%
Intermediate	R	$64.33 \pm 6.89\%$	$+73.31 \pm 8.93$ pp	0.00%
Uneven	A	$38.13 \pm 7.79\%$	$+56.96 \pm 9.09$ pp	0.00%
Uneven	R	$61.13 \pm 7.48\%$	$+60.55 \pm 9.62$ pp	0.00%

A: adaptive; R: round robin; pp: percentage points; SD: sample standard deviation across seeds. Estimate change is mean final BKT estimate minus the 20% prior.

TABLE VII
SIMULATION ESTIMATE SPREAD ACROSS 30 SEEDS

Profile	Policy	Weakest-topic estimate
Novice	Adaptive	$50.20 \pm 27.81\%$
Novice	Round robin	$28.16 \pm 17.16\%$
Intermediate	Adaptive	$96.13 \pm 8.72\%$
Intermediate	Round robin	$80.75 \pm 26.03\%$
Uneven	Adaptive	$38.26 \pm 25.51\%$
Uneven	Round robin	$42.30 \pm 28.64\%$

Entries are mean ± sample SD across 30 seeds.

Correct-answer probability was $a(1-s) + (1-a)g$, where a is the fixed topic ability.

Each profile used 30 seeds and 50 answers per run, grouped as ten consecutive blocks of five. No session time gaps or forgetting were simulated. The adaptive policy selected the current lowest-mastery topic through the implemented weak-topic selector. A round-robin baseline cycled across topics under the same question budget. Deterministic source-recall pools contained 83, 145 and 36 questions for graph representations, BFS and DFS, respectively; these were not live model-generated exams. Exact prompt fingerprints measured repetition within each run. Three profiles, two policies, 30 seeds and 50 answers produced 9,000 simulated answer events, not 9,000 unique questions.

The weakest-topic estimate is the minimum of the three final topic mastery estimates in each run. Table VII reports its mean and standard deviation. These are model estimates from fixed-ability simulations, not measured learning outcomes.

Adaptive weakest-topic selection rate is 100% by design; round-robin selection rates were 32.4%–36.1%, a consistency check rather than evidence of an advantage. Repetition occurred only when the 36-question DFS recall pool wrapped: two novice-adaptive seeds produced three and two repeated answers respectively (mean 0.33%). This is a pool-size effect, not a semantic novelty result.

The mean weakest-topic estimates favored adaptive selection for the novice (50.2% vs. 28.2%) and intermediate (96.1% vs. 80.8%) profiles. Comparing the paired seeds, these differences are about 4.8 and 3.3 standard errors respectively. They are simulated model estimates, not observed mastery or evidence of real learning. The uneven-profile differences in final mean mastery (3.6 pp) and weakest-topic estimate (4.0 pp) are not distinguishable at 30 paired seeds given their across-seed variation. Novice accuracy was similar under adaptive and round-robin policies (41.73% vs. 41.33%), while

mean estimated gains differed (+61.07 vs. +39.77 pp); this reflects BKT response to question order, not proof of better learning. The intermediate adaptive estimate approached the maximum possible gain of 80 pp. Simulated student abilities remain fixed while BKT assumes a positive learning transition.

These controlled runs demonstrate component state updates and selection behaviour only, not educational effectiveness or the complete live assessment workflow. Semantic novelty was not measured. The accompanying `benchmark_report.py` and `benchmark-results.json` preserve the primary questions, labels, seeds and outputs; the supplement is at `docs/evaluation/supplement/`. From the repository root, rerun:

```
python benchmark_report.py
python docs/evaluation/supplement/extended_checks.py
     --project .
python scripts/verify_report_evidence.py
```

These checks require installed project dependencies. The retrieval and simulation benchmark does not require Ollama or additional model downloads.

IX. Discussion

Local models support a workflow without mandatory paid inference, but extraction quality, depth and latency vary. Handwriting and notation can be misread; same-model checks may retain shared errors; and misconception labels remain hypotheses. Background processing improves responsiveness while long inference still takes time. The measured retrieval and simulation results are narrow; a broader benchmark is needed before claiming general accuracy or educational effectiveness.

X. Conclusion

ConceptFlow integrates multimodal reading, source-cited tutoring, revision tools, assessments and learner-aware progress in a local workflow. The prototype preserves source locations, completes supported assessment formats and stores reports for later review. Software checks, a labelled retrieval baseline and controlled simulations provide reproducible evidence of current behaviour, supporting continued improvements in question quality and personalised study guidance.

References

[1] S. Bai et al., *Qwen3-VL Technical Report*, arXiv:2511.21631, 2025. [Online]. Available: https://arxiv.org/abs/2511.21631

[2] SYSTRAN, *faster-whisper*. [Online]. Available: https://github.com/SYSTRAN/faster-whisper

[3] A. Radford et al., Robust speech recognition via large-scale weak supervision, in *Proc. ICML*, vol. 202, pp. 28492–28518, 2023.

[4] S. Robertson and H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] A. Yang et al., *Qwen2.5 Technical Report*, arXiv:2412.15115, 2024. [Online]. Available: https://arxiv.org/abs/2412.15115

[6] Ollama, *Ollama*. [Online]. Available: https://ollama.com

[7] P. Lewis et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[8] A. T. Corbett and J. R. Anderson, Knowledge tracing: Modeling the acquisition of procedural knowledge, *User Modeling and User-Adapted Interaction*, vol. 4, pp. 253–278, 1994, doi: 10.1007/BF01099821.

[9] ConceptFlow, *Source code and setup documentation*. [Online]. Available: https://github.com/silverflare08/Concept-Flow---adaptive-study-companion

[10] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, RAGAs: Automated evaluation of retrieval augmented generation, in *Proc. EACL: System Demonstrations*, pp. 150–158, 2024. [Online]. Available: https://aclanthology.org/2024.eacl-demo.16/